

Data Warehouse Overview

Star Schema Design

The greenfield_mp data warehouse uses a star schema design optimized for analytical queries and reporting. This design separates factual data (measures) from descriptive data (dimensions) to enable efficient querying and analysis.

Core Components

Fact Table: `fact_performance_reviews`

- **Grain:** One row per performance review
- **Primary Key:** `review_key` (BIGINT, AUTO_INCREMENT)
- **Surrogate Foreign Keys:**
 - `employee_key` → `dim_employee.employee_key`
 - `department_key` → `dim_department.department_key`
 - `project_key` → `dim_project.project_key`
 - `date_key` → `dim_date.date_key`
- **Degenerate Dimension:** `review_id` (OLTP performance review ID)
- **Measures:**
 - `performance_rating` (TINYINT, 1-4)
 - `job_satisfaction` (TINYINT, 1-4)
 - `environment_satisfaction` (TINYINT, 1-4)
 - `work_life_balance` (TINYINT, 1-4)
 - `percent_salary_hike` (TINYINT, 0-50)
 - `review_score` (DECIMAL(5,2), computed)

Dimension Tables

1. `dim_employee` (SCD Type 2)

- **Purpose:** Track employee changes over time
- **Primary Key:** `employee_key` (INT, AUTO_INCREMENT)
- **Business Key:** `employee_id` (INT)
- **Type 2 Attributes** (historical tracking):
 - `full_name` (VARCHAR(130))
 - `department_name` (VARCHAR(60))
 - `job_role` (VARCHAR(60))
 - `job_level` (TINYINT)
 - `monthly_income` (INT)
 - `start_date` (DATE)

- `end_date` (DATE)
- `is_current` (TINYINT(1))
- `change_reason` (VARCHAR(40))
- **Type 1 Attributes** (overwritten):
- `age` (TINYINT UNSIGNED)
- `marital_status` (VARCHAR(15))
- `education_level` (TINYINT)
- `education_field` (VARCHAR(40))
- `gender` (VARCHAR(10))
- `hire_date` (DATE)
- `attrition_flag` (TINYINT(1))
- `over_time_flag` (TINYINT(1))
- **Unique Constraint:** (employee_id, start_date)
- **Index:** (is_current, department_name)

2. `dim_department`

- **Purpose:** Department lookup
- **Primary Key:** `department_key` (INT, AUTO_INCREMENT)
- **Business Key:** `department_id` (INT)
- **Attributes:** `department_name` (VARCHAR(60))
- **Constraints:** UNIQUE on department_id and department_name

3. `dim_project`

- **Purpose:** Project lookup with unknown member
- **Primary Key:** `project_key` (INT, AUTO_INCREMENT)
- **Business Key:** `project_id` (INT)
- **Attributes:**
- `project_name` (VARCHAR(100))
- `department_name` (VARCHAR(60))
- `status` (VARCHAR(20))
- `priority` (VARCHAR(10))
- `budget` (DECIMAL(12,2))
- `start_date` (DATE)
- `end_date` (DATE)
- **Special Row:** Key -1, "No project" (for NULL project references)
- **Constraint:** UNIQUE on project_id

4. `dim_date`

- **Purpose:** Date dimension with smart key
- **Primary Key:** `date_key` (INT, yyyyymmdd format)
- **Attributes:**

- `full_date` (DATE)
- `day_of_month` (TINYINT)
- `day_name` (VARCHAR(10))
- `month_number` (TINYINT)
- `month_name` (VARCHAR(10))
- `quarter_number` (TINYINT)
- `year_number` (SMALLINT)
- `is_weekend` (TINYINT(1))
- **Constraint:** UNIQUE on `full_date`
- **Key Formula:** `YEAR(full_date) * 10000 + MONTH(full_date) * 100 + DAY(full_date)`

ETL Process

Overview

The ETL process moves data from the OLTP system to the data warehouse using stored procedures. The process is designed to be idempotent and incremental.

Stages

1. Date Dimension Load

```
CALL sp_load_dim_date('2015-01-01', '2030-12-31');
```

- Creates calendar dates from 2015-01-01 to 2030-12-31
- Uses digits cross join for efficient generation
- Populates all date attributes (day, month, quarter, year, weekend flag)

2. Department Dimension Load

```
CALL sp_load_dim_department();
```

- Type 1 update-then-insert strategy
- Updates existing rows if department names changed
- Inserts new departments
- Preserves surrogate keys

3. Project Dimension Load

```
CALL sp_load_dim_project();
```

- Type 1 update-then-insert strategy
- Updates project attributes

- Inserts new projects
- Maintains the "No project" unknown member (key -1)

4. Employee Dimension Load (SCD Type 2)

```
CALL sp_load_dim_employee_scd2(NULL); -- NULL = all employees
```

- Uses LEAD() window function to determine version end dates
- Two-step process: 1. Close/refresh existing rows (update end_date and is_current) 2. Insert new versions not yet in warehouse
- Handles both full refresh (NULL) and single employee refresh

5. Fact Table Load

```
CALL sp_load_fact_performance_reviews();
```

- Loads only new reviews not yet in fact table
- Uses anti-join pattern (LEFT JOIN ... WHERE IS NULL)
- Joins to dimension tables to get surrogate keys
- Uses employee version active on review date (SCD Type 2 join)
- Handles unknown member for NULL project references

Full ETL Pipeline

```
CALL sp_run_full_etl();
```

Executes all stages in correct order: 1. Date dimension 2. Department dimension 3. Project dimension 4. Employee dimension (SCD Type 2) 5. Fact table

Data Quality and Validation

Source Data Quality

- Raw data contains duplicates, inconsistent casing, and extra spaces
- ETL process handles cleaning:
 - `TRIM()` removes leading/trailing whitespace
 - `LOWER()` normalizes email case
 - `ROW_NUMBER()` removes duplicates
- Text-to-ID lookups ensure referential integrity

Transformation Logic

1. **Staging to OLTP:** - Remove duplicate employees (keep most recent hire_date) - Remove duplicate reviews (keep lowest review_id) - Clean text fields (TRIM, LOWER) - Map text

values to lookup IDs (departments, job_roles) - Convert string flags to boolean (Yes/No → 1/0)

2. **OLTP to Warehouse:** - Build date dimension from fixed range - Synchronize lookup tables (departments, job_roles) - Apply SCD Type 2 logic for employee history - Load only new fact records - Preserve degenerate dimensions (OLTP IDs)

Validation Checks

- Primary key uniqueness
- Foreign key referential integrity
- Check constraints (value ranges)
- Not null constraints on required fields
- Unique constraints on business keys

Performance Characteristics

Storage Efficiency

- Surrogate keys reduce storage vs. natural keys
- Dimension tables are relatively small
- Fact table is narrow (only measures and foreign keys)
- Date key as integer saves space vs. DATE type

Query Performance

- Star joins are efficient (small dimension tables)
- Surrogate keys enable efficient indexing
- Pre-aggregated data in dimensions
- Smart date key enables efficient date filtering

Indexing Strategy

Fact Table: - Primary key: `review_key` - Foreign key indexes: `employee_key`, `department_key`, `project_key`, `date_key` - Additional indexes: `date_key` (time-series), `department_key` (dept queries)

Dimension Tables: - Primary keys on surrogate keys - Unique constraints on business keys - Additional indexes: `dim_employee(is_current, department_name)`

Scalability

- Handles 100,000+ employees
- Supports 387,000+ performance reviews
- ETL procedures are set-based and efficient
- Incremental loading minimizes processing time

Usage Patterns

Common Analytical Queries

1. Time-Series Analysis

```
SELECT
    dt.year_number AS year,
    AVG(f.review_score) AS avg_score
FROM fact_performance_reviews f
JOIN dim_date dt ON f.date_key = dt.date_key
GROUP BY dt.year_number
ORDER BY dt.year_number;
```

2. Department Analysis

```
SELECT
    d.department_name,
    COUNT(*) AS review_count,
    AVG(f.review_score) AS avg_score
FROM fact_performance_reviews f
JOIN dim_department d ON f.department_key = d.department_key
GROUP BY d.department_name
ORDER BY avg_score DESC;
```

3. Employee History (SCD Type 2)

```
SELECT
    employee_key,
    employee_id,
    full_name,
    department_name,
    job_role,
    start_date,
    end_date,
    is_current
FROM dim_employee
WHERE employee_id = 12345
ORDER BY start_date;
```

4. Top Performers by Department

```
WITH employee_scores AS (
    SELECT
        e.employee_id,
        e.full_name,
        d.department_name,
        AVG(f.review_score) AS avg_score
    FROM fact_performance_reviews f
    JOIN dim_employee e ON f.employee_key = e.employee_key
    JOIN dim_department d ON f.department_key = d.department_key
    GROUP BY e.employee_id, e.full_name, d.department_name
),
```

```
ranked AS (  
    SELECT  
        *,  
        DENSE_RANK() OVER (PARTITION BY department_name ORDER BY avg_score DESC) AS  
        dept_rank  
    FROM employee_scores  
)  
SELECT department_name, employee_id, full_name, avg_score, dept_rank  
FROM ranked  
WHERE dept_rank <= 5  
ORDER BY department_name, dept_rank;
```

Maintenance and Operations

Backup and Recovery

- Full database backups recommended weekly
- Transaction logs for point-in-time recovery
- ETL procedures are re-runnable for recovery

Monitoring

- Row counts in each table
- ETL procedure execution times
- Data freshness (last load timestamp)
- Query performance metrics

Schema Evolution

- Adding new dimensions: Create table, update ETL
- Adding new measures: Alter fact table, update ETL
- Changing attribute types: Use ALTER TABLE with care
- Versioning: Track schema version in database

Comparison to Alternatives

Why Star Schema?

- **Simplicity:** Easy to understand and query
- **Performance:** Optimized for common analytical patterns
- **Flexibility:** Easy to add new dimensions or facts
- **Maintainability:** Clear separation of concerns

Compared to Snowflake

- Star schema denormalizes dimensions for query performance
- Snowflake normalizes dimensions further (more joins, less storage)

- Chose star schema for simplicity and query speed

Compared to 3NF

- 3NF minimizes redundancy but requires complex joins
- Star schema denormalizes for read performance
- Appropriate for read-heavy analytical workloads

Data Lineage

```
Source System (CSV)
↓
Data Synthesizer (Python)
↓
Staging Tables (MySQL)
↓
OLTP Database (Normalized)
↓
ETL Stored Procedures
↓
Data Warehouse (Star Schema)
↓
Analytics Layer (SQL Queries)
↓
Presentation Layer (Streamlit)
```

Business Rules Enforced

In Warehouse

1. **Employee Versioning:** Each review links to correct employee version
2. **Date Validity:** All dates within predefined range (2015-2030)
3. **Referential Integrity:** All foreign keys point to valid dimension rows
4. **Unknown Member Handling:** NULL projects map to "No project" dimension
5. **SCD Type 2 Integrity:** Proper start/end date sequencing

In ETL Process

1. **Idempotency:** Procedures can be re-run without side effects
2. **Incremental Loading:** Only new/changed data processed
3. **Data Cleaning:** Text normalization and duplicate removal
4. **Validation:** Referential integrity checks during load

Future Enhancements

1. Partitioning

- Partition fact table by date for improved query performance
- Enable partition pruning for date-range queries

2. Aggregation Tables

- Pre-aggregate common queries (monthly department stats)
- Improve dashboard load times

3. Columnar Storage

- Consider columnar storage for fact table (if MySQL variant supports)
- Improve compression and scan performance

4. Data Vault

- Consider Data Vault 2.0 for more complex historical tracking
- Better audit trail and flexibility

5. Real-time Capabilities

- Change data capture (CDC) for near-real-time updates
- Streaming ingestion for immediate availability

Conclusion

The greenfield_mp data warehouse implements a classic star schema with SCD Type 2 for employee history. The design balances simplicity, performance, and maintainability while providing the analytical capabilities needed for HR analytics. The ETL process ensures data quality and consistency, and the architecture supports the reporting requirements of the Streamlit application.